

| 第十周作业 -2

| 第十周作业 -2

| 提高作业

课程提供三张不同的足球图片：`ball_01.png`、`ball_02.png`、`ball_03.png`。要求：自行编写 Python 脚本，对每张图分别完成目标检测，输出足球的 bbox 和质心坐标。

| 2.1 编写检测脚本

自行编写 Python 脚本（可以一张图一个脚本，也可以写成统一入口），使用经典图像处理算法，参考以下步骤（注意：可以调整顺序或多层叠加）：

读取图片 → 预处理（滤波）→ 颜色空间转换 → 阈值分割 → 形态学处理 → 轮廓提取 → 输出结果

要求：

1. 每张图自行选择颜色空间、滤波方法并自行调参：三张图颜色不同，可以根据图片特点选择更合适的方法，提交 Python 代码；并在代码注释或说明中写明为什么选择这种滤波方法；
2. 给出每张图片的处理算法流程图，说明清楚算法设计思路；
3. 脚本最终输出格式：
 - 终端打印 `bbox=(x, y, w, h)` 和 `centroid=(cx, cy)`
 - 保存一张带有 bbox 矩形框和质心十字标记的可视化结果图
4. 若某张图片无法识别或识别错误，请给出分析思考依据，并认为可能是哪里存在问题。

编写脚本见最后代码部分，代码说明如下：

1. 预处理：保边降噪 (Step 1)
使用双边滤波 (`cv2.bilateralFilter`) 替代传统的高斯模糊。双边滤波可以在抹平场地噪点（如草皮碎屑）的同时，完好地保留足球黑白斑块之间的锐利边缘，为后续的纹理提取打下基础。
2. 双路并行分割 (Step 2 & Step 3)
脚本在此处兵分两路，分别提取目标的“宏观轮廓”和“微观纹理”：
 - 第一路 (找大轮廓 - HSV)：将图像转入 HSV 空间，锁定并剔除绿色的草地背景。剩下的非绿区域（包含足球、赛场白线、其他障碍物）被作为目标候选区，提取出它们的外轮廓。
 - 第二路 (提纹理 - 自适应阈值)：在灰度通道上，放弃了全局阈值 (Otsu)，改用局部自适应阈值 (**Adaptive Thresholding**)。这种方法对光照极度鲁棒，无论足球处于刺眼的强光下还是处于球员的阴影中，都能稳定剥离出表面的黑白斑块纹理。
3. 交叉融合与特征验证 (Step 4)
这是甄别足球的核心环节。程序遍历第一路找到的所有候选轮廓，对其进行严格的层层筛选：
 - 初筛 (形态过滤)：通过检测外接矩形的长宽比 (Aspect Ratio)，排除掉过于细长的干扰物（默认保留比例在 0.4 到 1.6 之间的物体）。
 - 终判 (黑白比例交叉验证)：将候选轮廓作为“遮罩”，去截取第二路生成的自适应纹理二值图。统计轮廓内部黑、白像素的比例。
 - 赛场白线、白色衣服往往呈现极端的白（白比例趋近 1.0）。
 - 深色阴影、纯黑物体呈现极端的黑。
 - 真正的足球拥有经典的黑白相间拼块，其白/黑比例一定是均衡的（被设定在 `0.2` 到 `0.85` 之间）。借此瞬间排除几乎所有的伪目标。
4. 规范化数据输出与可视化
当目标通过所有严苛验证被确认为足球后，脚本会：
 - 利用图像矩 (`cv2.moments`) 精准计算出足球中心位置。
 - 终端打印：严格输出 `bbox=(x, y, w, h)` 和 `centroid=(cx, cy)` 供下游机器人或控制程序调用。
 - 图像可视化：在最终图片上绘制显眼的绿色 Bounding Box，并在中心处打上红色的十字追踪标记 (Cross Marker)，供开发者直观检验。
5. 三张图最后的可视化结果图如下：



2.2 分析与总结

回答以下问题：

- 三张图中，哪张最难检测？你认为困难的主要原因是什么（背景干扰、颜色与环境色接近、光照不均匀，还是其他原因）？
- 在你的实验中，你为三张图选择了哪些滤波方法？选择依据是什么？实际效果是否符合预期？
- 如果足球颜色的 H 值和背景局部 H 值高度重叠（比如红色足球 + 红色广告牌背景），仅靠 HSV 阈值分割还够用吗？你会怎么改进？（思路即可，不要求实现）

- 第一张图较难检测，原因是草地上有大范围的白色干扰以及顶部的太阳光照射，这些都干扰了足球的特征识别。
- 使用了双边滤波 (**cv2.bilateralFilter**) 替代传统的高斯模糊。双边滤波可以在抹平场地噪点（如草皮碎屑）的同时，完好地保留足球黑白斑块之间的锐利边缘，为后续的纹理提取打下基础。效果基本符合需求。
- 当目标颜色与背景颜色高度重叠（如红色足球遇到了红色的广告牌或红色的球衣）时，仅靠 **HSV** 颜色阈值分割是绝对不够用的。HSV 只能告诉你“这里有一片红色”，但无法区分这片红色的语义归属，这会导致生成的掩膜 (Mask) 中足球和背景连成一片，彻底失效。
为了解决颜色混淆的问题，我们需要从“单一颜色依赖”升级为“多模态特征融合”。以下是几种经典的改进思路：

- 引入强几何形状约束 (Shape Features)

既然颜色不可靠，我们就用物理形状来区分：

- 圆度检测 (Circularity)**：即使红色连成一片，在进行边缘提取 (Canny) 后，红色广告牌通常是直线边缘或巨大的矩形，而足球是一个明确的圆。可以利用霍夫圆变换 (**Hough Circle Transform**) 直接锁定圆形边缘。
- 轮廓几何分析**：在寻找轮廓后，严格计算外接矩形的长宽比（必须接近 1）、凸性 (Convexity) 和圆度 ($4\pi \times \text{Area} / \text{Perimeter}^2$)，利用几何过滤掉广告牌。

- 引入纹理与内部特征 (Texture Features)

- 表面缝线/斑块特征**：红色足球表面通常也有接缝线条、品牌 Logo 或是不同深浅红色的拼接。通过计算局部区域的梯度 (Sobel) 或使用角点检测 (**Shi-Tomasi**)，可以发现足球区域有密集的纹理变化，而纯红色的广告牌通常纹理单一（除非上面写满了字）。
- LBP (局部二值模式)**：提取目标区域的 LBP 特征直方图，足球的球面反光纹理和广告牌的平面纹理在数学表达上是截然不同的。

- 利用动态与时空信息 (Motion & Temporal)

如果是处理视频流，运动信息是破局的关键：

- 背景建模 (Background Subtraction)**：广告牌是静止的，足球是运动的。使用混合高斯模型 (MOG2) 或 KNN 背景减除算法，能够直接把“正在运动的红色球”从“静止的红色广告牌”中抠出来。
- 光流法 (Optical Flow)**：追踪画面中像素的运动矢量，运动方向和速度与背景不一致的红色色块，即为足球。

- 引入深度信息 (Depth)

- RGB-D 相机**：如果机器人配备了双目视觉或深度相机（如 RealSense、Kinect），可以直接通过深度图 (**Depth Map**) 把背景剔除。无论颜色多么相似，广告牌在 3 米外，足球在 1 米处，在深度空间上它们是完全隔离的。

- 降维打击：深度学习 (Deep Learning)

- 摒弃传统视觉中手动设置规则的思路，直接引入轻量级目标检测模型（如 **YOLOv8-nano**）。深度学习模型提取的是高级语义特征（例如球的球面光影、拼接缝、整体轮廓），它“认识”什么是足球，根本不会被后面的红色广告牌所迷惑。

提高作业提交要求

整理为 一份 PDF，包含：

- 三张图各自的检测脚本代码（或统一脚本 + 说明）
- 三张图各自使用的颜色空间、滤波方法、阈值参数，及简短调参说明
- 三张图各自的最终可视化结果图（带 bbox + 质心）
- 2.2 三道分析题的回答

代码

```
"""足球检测演示脚本：双路并行阈值分割。
```

基于以下流程：

双边滤波（保边去噪）

→ 阈值分割（双路并行）：

- 第一路：HSV 过滤掉绿色（定位大轮廓，包含足球+白线）
- 第二路：提取灰度/V通道，使用 Otsu 大律法二值化（验证足球黑白纹理）

→ 轮廓结合与特征验证

```
"""
```

```
from __future__ import annotations
```

```
import argparse
```

```
import sys
```

```
from pathlib import Path
```

```
import cv2
```

```
import numpy as np
```

```
HERE = Path(__file__).resolve().parent
```

```
DEFAULT_IMAGE = HERE / "images" / "01_ball_clean.png"
```

```
# 草地的绿色 HSV 范围
```

```
HSV_GREEN_LOWER = np.array([15, 15, 20], dtype=np.uint8)
```

```
HSV_GREEN_UPPER = np.array([85, 255, 255], dtype=np.uint8)
```

```
MAX_DISPLAY_W = 640
```

```
def _fit(image: np.ndarray) -> np.ndarray:
    """等比缩放图像以适应显示."""
    h, w = image.shape[:2]
    if w <= MAX_DISPLAY_W:
        return image
    scale = MAX_DISPLAY_W / w
    return cv2.resize(image, (MAX_DISPLAY_W, int(h * scale)), interpolation=cv2.INTER_AREA)

def _show(title: str, image: np.ndarray, out_dir: Path, filename: str) -> None:
    """弹窗显示并保存截图."""
    out_dir.mkdir(parents=True, exist_ok=True)
    cv2.imwrite(str(out_dir / filename), image)
    cv2.imshow(title, _fit(image))

def _wait_and_close() -> None:
    print("    [按任意键进入下一步, 或按 'q' 键退出程序]")
    key = cv2.waitKey(0) & 0xFF
    cv2.destroyAllWindows()
    if key == ord('q'):
        print("\n[退出] 用户按下了 'q' 键, 提前终止流水线。")
        sys.exit(0)

def step0_read_and_show(image_path: Path, out_dir: Path) -> np.ndarray:
    print("\n[Step 0] 读图")
    img = cv2.imread(str(image_path))
    if img is None:
        # 生成测试图
        img = np.zeros((480, 640, 3), dtype=np.uint8)
        img[:] = (50, 200, 50) # 绿色背景
        cv2.line(img, (0, 240), (640, 240), (255, 255, 255), 10) # 白线
        cv2.circle(img, (150, 150), 40, (255, 255, 255), -1) # 纯白圆干扰
        # 足球 (黑白相间)
        cv2.circle(img, (450, 240), 50, (255, 255, 255), -1)
        cv2.circle(img, (450, 240), 15, (0, 0, 0), -1)
        for i in range(5):
            cv2.circle(img, (450 + int(30 * np.cos(i)), 240 + int(30 * np.sin(i))), 12, (0, 0, 0), -1)

    _show("Step0 Original", img, out_dir, "step0_original.png")
    _wait_and_close()
    return img

def step1_bilateral_filter(img: np.ndarray, out_dir: Path) -> np.ndarray:
    print("\n[Step 1] 双边滤波 — 保边去噪")
    # d=9, sigmaColor=75, sigmaSpace=75
    # 相比高斯滤波, 双边滤波能更好地保留足球边缘和斑块边缘
    blurred = cv2.bilateralFilter(img, 9, 75, 75)

    pair = np.hstack([img, blurred])
    _show("Step1 Bilateral Filter", pair, out_dir, "step1_bilateral.png")
    _wait_and_close()
    return blurred

def step2_path1_hsv_contour(blurred: np.ndarray, out_dir: Path) -> np.ndarray:
    print("\n[Step 2 - 第一路] HSV 过滤草地 — 提取大轮廓候选区 (足球+白线)")
    hsv = cv2.cvtColor(blurred, cv2.COLOR_BGR2HSV)
    green_mask = cv2.inRange(hsv, HSV_GREEN_LOWER, HSV_GREEN_UPPER)
    non_green_mask = cv2.bitwise_not(green_mask)

    # 形态学操作去除小碎屑, 连接断裂处
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (13, 13))
    non_green_mask = cv2.morphologyEx(non_green_mask, cv2.MORPH_OPEN, kernel)
    non_green_mask = cv2.morphologyEx(non_green_mask, cv2.MORPH_CLOSE, kernel)

    _show("Path1: Non-Green Mask", non_green_mask, out_dir, "step2_path1_mask.png")
    _wait_and_close()
    return non_green_mask

def step3_path2_adaptive_texture(blurred: np.ndarray, out_dir: Path) -> np.ndarray:
    print("\n[Step 3 - 第二路] 灰度提取 + 自适应阈值 (Adaptive Thresholding) — 抵抗光照不均")
    gray = cv2.cvtColor(blurred, cv2.COLOR_BGR2GRAY)

    # 抛弃全局阈值(Otsu), 改用局部自适应阈值
    # 这在处理“有一半在阴影里的足球”时, 能保持纹理提取的稳定性
    adaptive_mask = cv2.adaptiveThreshold(
        gray, 255,
```



```
        cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY,
        11, 2
    )

    _show("Path2: Adaptive Binary Mask", adaptive_mask, out_dir, "step3_path2_adaptive.png")
    _wait_and_close()
    return adaptive_mask

def step4_combine_and_verify(img: np.ndarray, contour_mask: np.ndarray, texture_mask: np.ndarray, out_dir: Path) -> None:
    print("\n[Step 4] 双路合并: 定位大轮廓 -> 利用自适应纹理掩膜验证足球")

    # 1. 从第一路（非绿区域）寻找所有的轮廓
    contours, _ = cv2.findContours(contour_mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    vis = img.copy()

    for i, c in enumerate(contours):
        area = cv2.contourArea(c)
        if area < 500: # 过滤掉太小的区域
            continue

        x, y, w, h = cv2.boundingRect(c)
        aspect_ratio = float(w) / h

        # 简单形态过滤: 足球的包围盒长宽比应接近 1
        if 0.4<= aspect_ratio <= 1.6:
            # 创建该轮廓的局部掩膜
            c_mask = np.zeros_like(contour_mask)
            cv2.drawContours(c_mask, [c], -1, 255, -1)

            # 2. 纹理验证: 计算该轮廓在第二路（自适应二值图）中的黑白比例
            # 将自适应二值图与当前轮廓掩膜取交集
            roi_otsu = cv2.bitwise_and(texture_mask, texture_mask, mask=c_mask)

            # 统计轮廓内的白色像素点(代表亮色区域/白斑)和黑色像素点(代表暗色区域/黑斑)
            total_pixels = cv2.countNonZero(c_mask)
            white_pixels = cv2.countNonZero(roi_otsu)
            black_pixels = total_pixels - white_pixels

            white_ratio = white_pixels / total_pixels if total_pixels > 0 else 0

            print(f"  候选轮廓 {i}: 面积={area:.0f}, 长宽比={aspect_ratio:.2f}, 白/黑比例: {white_ratio:.2f} / {1-white_ratio:.2f}")

            # 3. 核心判别逻辑: 足球通常既有白色也有黑色
            # 纯白线或纯白圆的 white_ratio 会接近 1.0 (比如 > 0.9)
            # 阴影或纯黑物体的 white_ratio 会接近 0.0 (比如 < 0.1)
            # 足球黑白相间, 通常在 0.2 ~ 0.8 之间
            if 0.2 < white_ratio < 0.85:
                print("    -> 【验证通过】 判定为足球! ")

                # 计算质心
                m = cv2.moments(c)
                if m["m00"] != 0:
                    cx = int(m["m10"] / m["m00"])
                    cy = int(m["m01"] / m["m00"])
                else:
                    cx, cy = x + w // 2, y + h // 2

                # 按照要求输出终端打印信息
                print(f"    -> bbox=(x={x}, y={y}, w={w}, h={h})")
                print(f"    -> centroid=({cx}, {cy})")

                # 可视化结果图: 绘制 bbox 矩形框和质心十字标记
                cv2.rectangle(vis, (x, y), (x+w, y+h), (0, 255, 0), 3)
                cv2.drawMarker(vis, (cx, cy), (0, 0, 255), cv2.MARKER_CROSS, 20, 2)
                cv2.putText(vis, "Soccer", (x, y-10), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)
            else:
                print("    -> 【验证失败】 纯色物体（线段/干扰圆）被排除")
                cv2.rectangle(vis, (x, y), (x+w, y+h), (0, 0, 255), 2)
    else:
        # 长宽比严重失调（比如长长的白线）
        cv2.rectangle(vis, (x, y), (x+w, y+h), (0, 0, 255), 1)

    _show("Step4 Final Detection", vis, out_dir, "step4_final.png")
```

```
_wait_and_close()

def main() -> None:
    parser = argparse.ArgumentParser(description="双路并行阈值分割检测足球")
    parser.add_argument("--image", type=Path, default=DEFAULT_IMAGE, help="输入图片路径")
    args = parser.parse_args()

    image_path = args.image if args.image.is_absolute() else (HERE / args.image).resolve()
    out_dir = HERE / "outputs" / "parallel_demo"
    print(f"[pipeline] image={image_path.name}  outputs={out_dir}")

    img = step0_read_and_show(image_path, out_dir)
    blurred = step1_bilateral_filter(img, out_dir)
    contour_mask = step2_path1_hsv_contour(blurred, out_dir)
    texture_mask = step3_path2_adaptive_texture(blurred, out_dir)
    step4_combine_and_verify(img, contour_mask, texture_mask, out_dir)

    print("\n[pipeline] 双路并行检测完成.")

if __name__ == "__main__":
    main()
```